

Table of contents

Série 11 : Vérification de Modèles avec Automates de Büchi et LTL	1
Méthodes	1
Construction d'Automates de Büchi (BA) à partir d'Expressions -Régulières .	1
Opérations sur les Automates de Büchi	1
Traduction entre Propriétés, Expressions -Régulières, LTL et Automates de Büchi	2
Vérification de Satisfiabilité et de Validité pour les Formules LTL	2
Vérification de Modèles (Model Checking) par Produit Synchrone	2
Manipulation des Connecteurs LTL Étendus	3
Rappel	3
Exemples Illustratifs	3
Exemple 1 : Traduction d'une Propriété en BA et Expression -Régulière	3
Exemple 2 : Opération de Concatenation FA et BA	4
Exemple 3 : Model Checking par Produit Synchrone	5
Synthèse	5

Série 11 : Vérification de Modèles avec Automates de Büchi et LTL

Méthodes

Construction d'Automates de Büchi (BA) à partir d'Expressions -Régulières

- **Objectif** : Représenter un langage de mots infinis (-langage) décrit par une expression -régulière sous forme d'automate de Büchi.
- **Méthode** : Appliquer des constructions de base pour les opérateurs de concaténation, union, étoile de Kleene et omega-puissance ($\cdot^\omega$), puis les combiner.

Opérations sur les Automates de Büchi

- **Concatenation d'un Automate Fini (FA) et d'un BA** :
 1. Conserver les états des deux automates A (FA) et B (BA).
 2. L'état initial est celui de A .
 3. Les états acceptants (finaux) sont ceux de B .
 4. Conserver toutes les transitions existantes.
 5. Pour chaque transition de A allant vers un de ses états finaux, ajouter une transition étiquetée de même depuis l'origine vers l'état initial de B .
- **Omega-puissance d'un Automate Fini (A^ω)** :
 1. Conserver les états de l'automate fini A .

2. L'état initial est celui de A .
3. Les états acceptants sont ceux qui sont accessibles depuis eux-mêmes via un cycle (i.e., font partie d'une composante fortement connexe accessible).
4. Toutes les transitions sont conservées. L'automate ainsi obtenu accepte les exécutions infinies qui traversent indéfiniment au moins un état acceptant.

Traduction entre Propriétés, Expressions ω -Régulières, LTL et Automates de Büchi

- **Objectif** : Exprimer une propriété décrite en langage naturel sous différentes formes formelles équivalentes.
- **Processus** :
 1. Comprendre la sémantique de la propriété (mots infinis acceptés vs rejetés).
 2. Proposer une expression ω -régulière qui capture cette sémantique.
 3. Construire un BA qui reconnaît cette expression.
 4. (Si possible) Fournir une formule LTL équivalente.

Vérification de Satisfiabilité et de Validité pour les Formules LTL

- **Satisfiabilité** : Existe-t-il au moins un mot infini (une exécution) qui satisfait la formule ϕ ? Méthode : Proposer un contre-exemple concret w tel que $w \models \phi$.
- **Validité** : La formule ϕ est-elle vraie pour tous les mots infinis ? Méthode : Prouver la validité par raisonnement sémantique, ou montrer qu'elle n'est pas valide en exhibant un contre-exemple w tel que $w \not\models \phi$.

Vérification de Modèles (Model Checking) par Produit Synchrone

- **Objectif** : Vérifier si tous les comportements (mots infinis) d'un système A satisfont une propriété ϕ ($A \models \phi$).
- **Méthode** :
 1. Négation : Construire l'automate de Büchi $B_{\neg\phi}$ pour la négation de la propriété.
 2. Produit Synchrone : Construire l'automate produit $A \times B_{\neg\phi}$ qui reconnaît l'intersection des langages $L(A) \cap L(\neg\phi)$.
 3. Analyse des SCC : Chercher dans le graphe produit une Composante Fortement Connexe (SCC) qui soit accessible depuis l'état initial et qui contienne **au moins un état acceptant** de $B_{\neg\phi}$.
 - Si une telle SCC existe : $L(A) \cap L(\neg\phi) \neq \emptyset$, donc $A \not\models \phi$. Un contre-exemple est un mot dans cette SCC.
 - Si aucune telle SCC n'existe : $L(A) \cap L(\neg\phi) = \emptyset$, donc $A \models \phi$.

Manipulation des Connecteurs LTL Étendus

- **Weak Until** (pWq) : “ p reste vrai jusqu’à ce que q devienne vrai, sans garantie que q arrive”. Formellement : $\sigma \models pWq \iff (\sigma \models p\mathcal{U}q) \vee (\sigma \models \Box p)$.
- **Infinitely Often** ($F^\infty p$) : “ p est vrai une infinité de fois”. Formellement : $\sigma \models F^\infty p \iff \forall i \geq 0, \exists j \geq i, \sigma^j \models p$. Peut s’exprimer en LTL standard par $\Box \diamond p$.

Rappel

- **Automate de Büchi** : Automate sur des mots infinis où un mot est accepté s’il existe une exécution qui visite **un état acceptant (final) une infinité de fois**.
- **Expression -Régulière** : Généralise les expressions régulières aux langages infinis. Les opérateurs de base sont la concaténation, l’union, l’étoile de Kleene $*$ (répétition finie) et l’opérateur omega ω (répétition infinie).
- **Sémantique LTL** : Une formule LTL est interprétée sur un mot infini $\sigma = \sigma_0\sigma_1\sigma_2\dots$
 - $\sigma \models \circ\phi \iff \sigma^1 \models \phi$
 - $\sigma \models \Box\phi \iff \forall i \geq 0, \sigma^i \models \phi$
 - $\sigma \models \diamond\phi \iff \exists i \geq 0, \sigma^i \models \phi$
 - $\sigma \models \phi\mathcal{U}\psi \iff \exists i \geq 0$ tel que $\sigma^i \models \psi$ et $\forall 0 \leq j < i, \sigma^j \models \phi$
- **Produit Synchrone** : Étant donné $A = (Q_A, \Sigma, \delta_A, q_{0A}, F_A)$ et $B = (Q_B, \Sigma, \delta_B, q_{0B}, F_B)$, le produit $A \times B$ a pour états $Q_A \times Q_B$, état initial (q_{0A}, q_{0B}) , et une transition $((s, t), a, (s', t'))$ si $(s, a, s') \in \delta_A$ et $(t, a, t') \in \delta_B$. Les états acceptants dépendent du type d’automate.

Exemples Illustratifs

Exemple 1 : Traduction d’une Propriété en BA et Expression -Régulière

Contexte : Exprimer la propriété “a response is eventually provided on each request”.

Application :

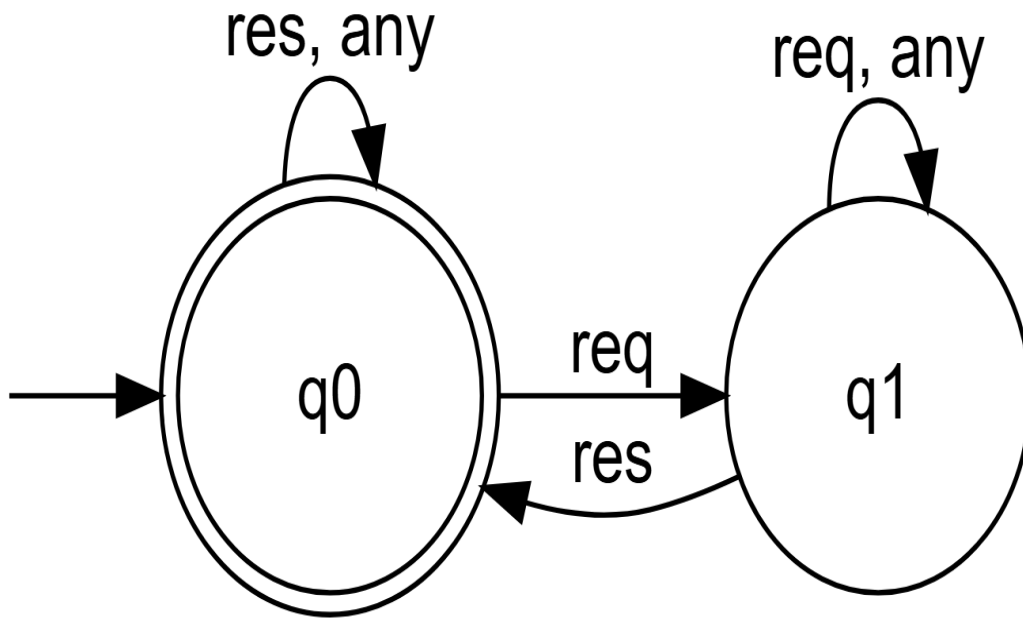
1. **Sémantique** : Après chaque occurrence de **req**, il doit finir par y avoir une occurrence de **res**. Un mot comme **(req any* res)[^]** est accepté. Un mot comme **req (any)[^]** (une requête jamais servie) est rejeté.
2. **Expression -Régulière** :

$$((\text{req} \cup \text{res} \cup \text{any})^* \cdot \text{res})^\omega \quad \cup \quad ((\text{req} \cup \text{res} \cup \text{any})^* \cdot \text{req} \cdot (\text{any})^* \cdot \text{res})^\omega$$

Une expression plus simple et équivalente serait $((\neg \text{req})^* \cdot \text{req} \cdot (\neg \text{res})^* \cdot \text{res})^\omega$

3. **Construction du BA** : L’automate a deux états principaux :

- q_0 : “En attente d’une requête ou rien de spécial”.
- q_1 : “Une requête a été émise et on attend la réponse”.
- Transition de q_0 à q_1 sur **req**.
- Transition de q_1 à q_0 sur **any** mais **res**.
- Transition de q_1 à q_0 sur **res** (la requête est servie).
- Toutes les autres transitions (e.g., **any** dans q_0 , **req** dans q_1) restent dans l’état courant.
- L’état initial est q_0 . **Tous les états sont acceptants** pour garantir que les requêtes en attente (q_1) doivent être quittées (en allant vers q_0 via **res**) pour que l’exécution soit acceptante (car il faut visiter un état acceptant une infinité de fois, on ne peut donc rester bloqué dans q_1).



Exemple 2 : Opération de Concatenation FA et BA

Contexte : Construire un BA pour $(ab + c)c^*(ab^*)^\omega$ à partir d’un FA pour $(ab + c)c^*$ et d’un BA pour $(ab^*)^\omega$.

Application :

1. Soit A le FA pour $(ab + c)c^*$. Il a un état initial i_A et (au moins) un état final f_A .
2. Soit B le BA pour $(ab^*)^\omega$. Il a un état initial i_B et des états acceptants F_B .
3. **Construction du BA pour la concaténation :**
 - États : Union disjointe des états de A et B .

- État initial : i_A .
 - États acceptants : F_B (ceux de B seulement).
 - Transitions : Toutes les transitions de A et de B sont conservées.
 - **Transitions ajoutées** : Pour **chaque** transition dans A de la forme (p, α, f_A) où f_A est un état final de A , on ajoute une **nouvelle** transition (p, α, i_B) vers l'état initial de B .
4. **Justification** : Pour être accepté, un mot doit d'abord être accepté par le préfixe fini reconnu par A (ce qui amène à un état final f_A), puis immédiatement commencer et satisfaire le suffixe infini reconnu par B . La transition ajoutée permet ce "passage de relais".

Exemple 3 : Model Checking par Produit Synchrone

Contexte : Vérifier si l'automate A (avec $L(A) = (bc)^\omega$) satisfait $\phi_2 = \Box(a \Rightarrow \Diamond c)$.

Application :

1. **Négation de la propriété** : $\neg\phi_2 = \neg\Box(a \Rightarrow \Diamond c) = \Diamond(a \wedge \Box\neg c)$.
2. **Construction de $B_{\neg\phi_2}$** :
 - Il cherche à capturer un moment où a est vrai et ensuite c est **toujours faux**.
 - L'automate montré a des états représentant "n'a pas encore vu $a \wedge \Box\neg c$ " (initial), "a vu a et maintenant vérifie $\Box\neg c$ " (acceptant).
 - Il reste dans l'état acceptant uniquement en lisant des lettres où c est faux.
3. **Produit Synchrone $A \times B_{\neg\phi_2}$** :
 - On couple les états de A (par exemple s_1, s_2, s_3) avec ceux de B .
 - L'automate A ne génère que les lettres **b** et **c**. Il ne génère **jamais** la lettre **a**.
4. **Analyse** : La prémisse a de la formule ϕ_2 n'est jamais vraie dans aucun mot de $L(A)$. Par conséquent, l'implication $a \Rightarrow \Diamond c$ est **vacuously true** à chaque position. La formule ϕ_2 est donc vraie sur tout mot de $L(A)$.
5. **Vérification par SCC** : Dans le produit, aucune exécution ne peut atteindre l'état acceptant de $B_{\neg\phi_2}$ car cela nécessiterait de lire un **a**. Donc, aucune SCC accessible ne contient d'état acceptant de $B_{\neg\phi_2}$. On conclut que $L(A) \cap L(\neg\phi_2) = \emptyset$, donc $A \models \phi_2$.

Synthèse

Cette série d'exercices couvre le cœur de la vérification de modèles basée sur les automates. Les compétences clés sont :

1. la manipulation formelle des ω -langages via des expressions et des automates de Büchi
2. la traduction entre différentes notations (LTL, expressions, automates)

3. l'utilisation d'opérations sur les automates pour construire des représentations complexes
4. l'application de l'algorithme central de model checking via le produit synchrone et la recherche de SCC.

La compréhension fine de la sémantique de LTL et de l'acceptation par Büchi est fondamentale pour toutes ces tâches.